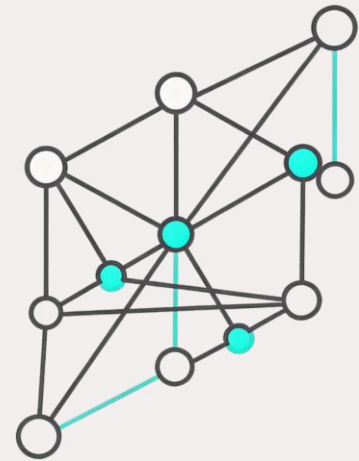


בעיות סיפוק אילוצים – חלק II

פרק | 6 סעיפים 4-5



חיפוש מקומי עבור CSPs

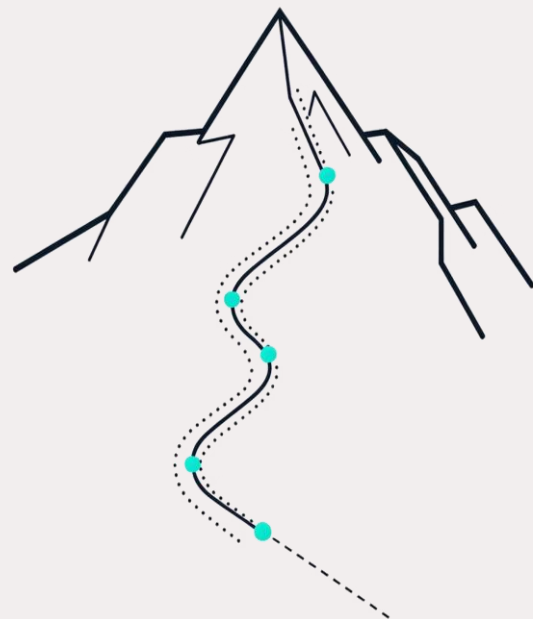
חיפוש מקומי (Local Search) הוא משפחת אלגוריתמים שמתחילים ממצב כלשהו ומנסים לשפר אותו בצעדים קטנים. בניגוד לחיפוש עץ מסורתי, הם שומרים רק על המצב הנוכחי – ללא זיכרון של המסלול.

Hill Climbing

מתקדמים תמיד לכיוון
הטוב ביותר, ללא נסיגה

Simulated Annealing

מאפשרים לפעמים צעדים
גרועים כדי להימנע
ממינימום מקומי



האם CSP מתאים לחיפוש מקומי?

לפני שנחיל חיפוש מקומי על בעיות CSP, עלינו לענות על שלוש שאלות מהותיות - שאלות עקרוניות שמתאימות לכל בעיית CSP

1

מצב התחלתי

מהו המצב ממנו נתחיל ?

2

ערך היוריסטי

כיצד נמדוד את איכות
המצב ?

3

פעולת עוקב

מה השינוי האפשרי ?

דוגמה: צביעת מפה של אוסטרליה

משתנים

WA, NT, Q, SA, NSW, V, T

אילוצים

כל שני אזורים שכנים חייבים

לקבל צבעים שונים



ניסוח חיפוש מקומי לCSP

מצב (State)

מוגדר על ידי ערכי

ההשמה הנוכחיים לכל

המשתנים

מצב התחלתי

השמה מלאה רנדומלית

לכל משתנה מוצמד ערך

כלשהו מתחום הערכים שלו.

פונקציית עוקב

שינוי הערך של **משתנה**

שמפר אילוץ לערך אחר

מתחום הערכים שלו.

מבחן המטרה

השמה מלאה וחוקית –

כל המשתנים מוגדרים ואף

אילוץ אינו מופר.

עלות מסלול

עלות של **1 לכל שינוי**

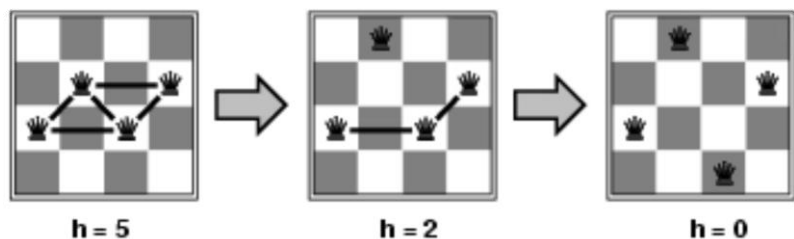
השמה – מונה את מספר

הצעדים שבוצעו.

דוגמה: בעיית 4-מלכות

הגדרת הבעיה

- **מצבים 4:** מלכות ב 4-עמודות 256 - מצבים אפשריים
- **פעולות:** הזזת מלכה בעמודה שלה
- **מבחן מטרה:** אין התקפות
- **הערכה** $h(n)$: מספר זוגות המלכות המתקיפות זו את זו



במצב התחלתי $h=5$ לאחר מהלך אחד
 $h=2$. לאחר מהלך נוסף $h=0$ פתרון!

חיפוש מקומי ל CSP שאלות מפתח

אלגוריתמים כמו Hill-Climbing ו-Simulated Annealing-עובדים עם **השמות מלאות**—לכל המשתנים יש ערך בכל עת. פעולה היא שינוי ההשמה של משתנה **יחיד** שמפר

שתי שאלות מרכזיות מנחות את תכנון האלגוריתם: 

בחירת ערך

איזה **ערך חדש** ניתן לאותו משתנה?

בחירת משתנה

כיצד נבחר **לאיזה משתנה** לשנות את ההשמה?

היוריסטיקת Min-Conflicts

ההיוריסטיקה **Min-Conflicts** מספקת תשובה אלגנטית לשאלת בחירת הערך: מתוך כל הערכים האפשריים למשתנה שנבחר, בוחרים את הערך שמפר את **המספר המינימלי של אילוצים** במצב הנוכחי.

פונקציית הערכה

$h(n)$ = סך כל האילוצים המופרים

.המטרה היא להגיע 0

לכל ערך v אפשרי, נחשב כמה אילוצים

יופרו אם נציב את v ונבחר את המינימלי.

בחירת משתנה

באופן **רנדומלי** – בוחרים משתנה שמפר

לפחות אילוץ אחד מתוך קבוצת

המשתנים הקונפליקטואליים.

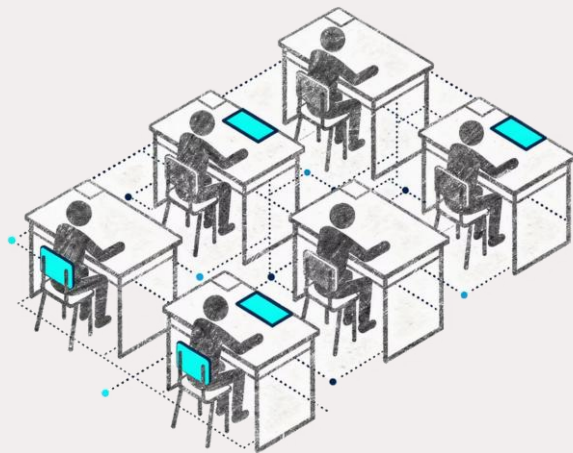
אלגוריתם MinConflicts

```
MinConflicts(csp, max_steps):  
    current ← an initial complete assignment for csp  
    for i = 1 to max_steps:  
        {if current is a solution for csp:  
            return current  
        X ← a randomly selected conflicted variable from Vars[csp]  
        val ← a value for X minimizing Conflicts(X, current, csp)  
        current = current with {X = val}  
    }  
    return failure
```

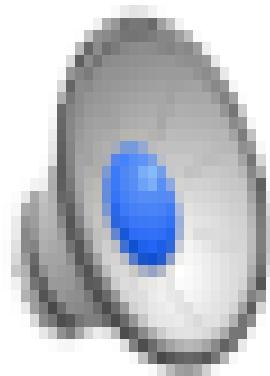
הדגמה: ישיבה בכיתה

בעיית ישיבה בכיתה : כיצד להושיב תלמידי בכיתה
כשיש העדפות ישיבה למשל, שני סטודנטים
מסוימים לא יושבים ליד אחד .
כיצד נגדיר הבעיה כבעיית סיפוק אילוצים?

<https://www.youtube.com/watch?v=-FKcPiMjkwE>

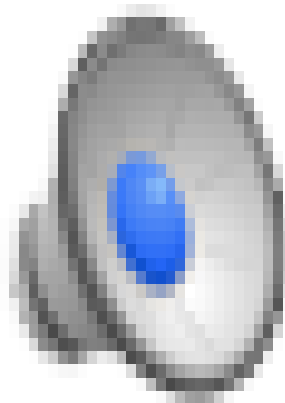


הדגמת Min-Conflicts: בעיית -חמלכות



הדגמה זו נוצרה על ידי **Dan Klein** ו**Pieter Abbeel**-עבור קורס CS188 באוניברסיטת UC Berkeley.

הדגמת Min-Conflicts צביעת מפה



יעילות חיפוש מקומי ל-CSPs

כאשר מתחילים ממצב **סביר**, חיפוש מקומי מציג ביצועים מרשימים במיוחד על בעיות CSP קשות:

10

דקות

לתזמון תצפיות טלסקופ החלל האבל
לעומת 3 שבועות בחיפוש מסורתי

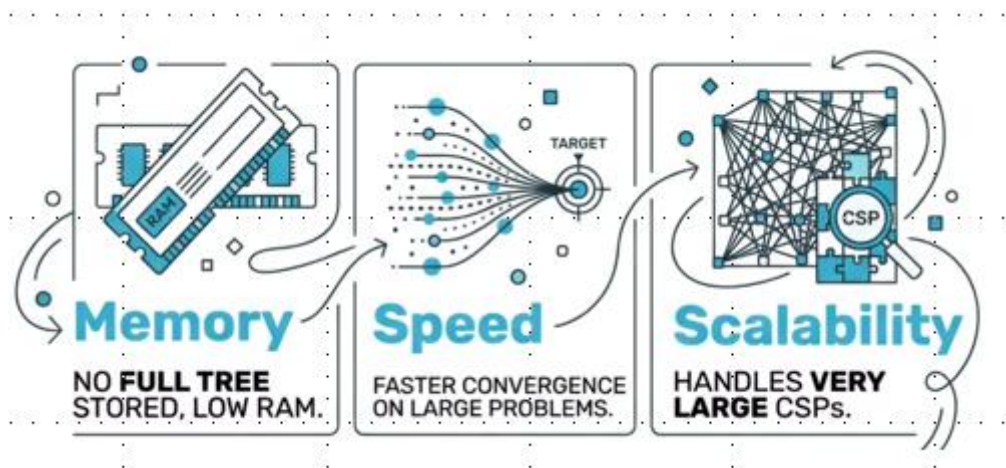
50

צעדים

לפתרון בעיית מיליון מלכות כמעט קבוע
ביחס לגודל!

✔ **יתרון: online** כאשר הבעיה משתנה תוך כדי ריצה, ניתן להתחיל מהפתרון הקודם ולמצוא פתרון חדש תוך שינויים מינימליים – יעיל בהרבה מחיפוש נסוג מאפס.

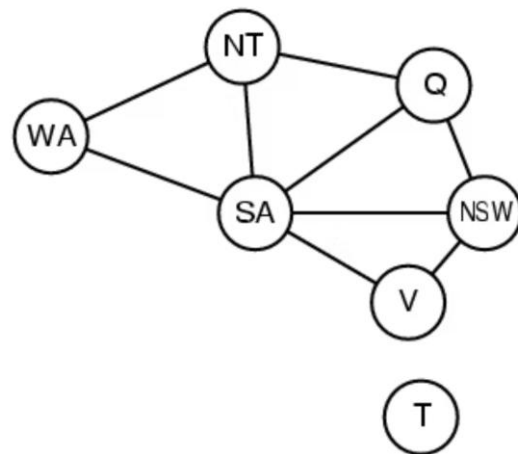
יעילות חיפוש מקומי לCSPs



עדיין מדובר בחיפוש מקומי ולכן ייתכן שיתקע ולא יוכל למצוא פתרון שמכבד את כל האילוצים. ✓

גרף האילוצים (Constraint Graph)

ניתן לייצג כל בעיית CSP כגרף שבו הקדקודים הם המשתנים והקשתות מייצגות אילוצים בין זוגות משתנים. ייצוג זה חושף את מבנה הבעיה ומאפשר לבצע אותו ליעול הפתרון.



שימו לב: **טסמניה (T)** מופיעה כקדקוד מבודד בגרף – אין לה קשתות לאזורים אחרים, מה שמשקף את היותה אי ללא גבול משותף ביבשת.

מבנה הבעיה – תת-בעיות בלתי תלויות

טסמניה אינה חלק מהיבשת – ולכן ההשמה שלה אינה משפיעה על שאר ההשמות ולא מושפעת מהן. זהו מקרה של **תת-בעיות בלתי תלויות**:

→ זיהוי רכיבי קשירות

בגרף האילוצים, כל **רכיב קשיר** הוא תת-בעיה בלתי תלויה שניתן לפתור בנפרד.

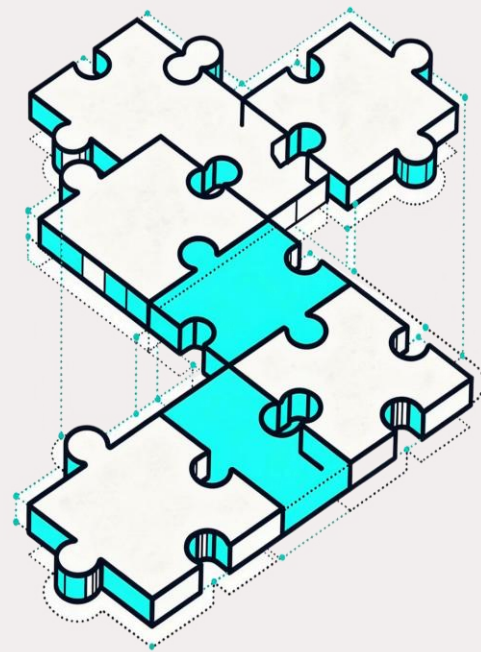
→ פתרון כל תת-בעיה

פותרים כל תת-בעיה **באופן עצמאי**, ללא תלות בתת-הבעיות האחרות.

→ שילוב הפתרונות

איחוד הפתרונות של כל תת-הבעיות הוא הפתרון הכולל לבעיה

המקורית.



מבנה הבעיה – ניתוח סיבוכיות

פירוק לתת-בעיות בלתי תלויות מוביל לחיסכון **אדיר** בזמן החישוב.

נניח n משתנים, תחום d ערכים, ופירוק לתת-בעיות עם c משתנים כל אחת, c קבוע:

פתרון כולל ללא פירוק

$O(d^n)$ אקספוננציאלי ב n

פתרון מפורק עם פירוק

$O(d^c \cdot n/c)$ לינארי ב n

דוגמה מספרית $n=80, d=2, c=20$:

זמן משוער (10M nodes/sec)

4 מיליארד שנה

0.4 שניות בלבד!

מספר פעולות

$2^{80} \approx 10^{24}$

$4 \times 2^{20} \approx 4M$

גישה

פתרון כולל

פתרון מפורק